Maven Overview

Project Directory Structure

Key Directories:

- `src/main/java` → Java source code
- `src/main/resources` → Configuration, properties, non-Java artifacts
- `src/test/java` → Test source code
- `src/test/resources` → Test resources

Project Configuration (`pom.xml`)

- Defines **project metadata, dependencies, build plugins, and lifecycles**

Parent:

```xml
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>4.0.3</version>
  <relativePath/>
</parent>
```

Project Info:

```xml
<groupId>com.nbicocchi</groupId>
<artifactId>product-service-no-db</artifactId>
<version>0.0.1-SNAPSHOT</version>
<description>Demo product for Spring Boot</description>
```

Properties:

```xml
<properties>
  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  <java.version>21</java.version>
</properties>
```

Dependencies:

- Manage libraries and frameworks for your project

```xml
<dependencies>
  <dependency>
    <groupId>org.projectlombok</groupId>
    <artifactId>lombok</artifactId>
    <optional>true</optional>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
  </dependency>
</dependencies>
```

- View transitive dependencies:

```
mvn dependency:tree
```

Build Plugins

Customize how Maven builds and packages the project

Spring Boot Maven Plugin:

- Build and run Spring Boot apps
- Packages executable **fat jar**

```xml
<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
    </plugin>
  </plugins>
</build>
```

Maven Lifecycles

- Maven organizes the build process into **three main lifecycles**:

Lifecycle	Purpose
Default	Build, test, and package the project
Clean	Remove previous build outputs
Site	Generate project documentation and reports

Default Lifecycle

- Most commonly used lifecycle
- Key Phases:
  - **validate** → check project is correct
  - **compile** → compile source code
  - **test** → run unit tests
  - **package** → create JAR/WAR
  - **verify** → run quality checks
  - **install** → install to local repo
  - **deploy** → deploy to remote repo

Example:

```
mvn package
```

Clean Lifecycle

- Cleans the project build

Key Phases:

- **pre-clean** → pre-clean steps
- **clean** → remove `target` directory
- **post-clean** → post-clean steps

Site Lifecycle

- Generates project documentation

Key Phases:

- **pre-site** → before generating site
- **site** → generate site
- **post-site** → after generating site
- **site-deploy** → deploy site to server

Running the Project

Using Spring Boot Maven Plugin

```
mvn spring-boot:run
```

- Builds and runs the app locally
- **Not recommended for production**
  - Requires source code on server
  - Cold start is slow

Running as an Executable Jar

- Pre-packaged **fat jar** includes all dependencies

```
mvn clean package -Dmaven.skip.test=true
java -jar target/product-service-0.0.1-SNAPSHOT.jar
```

Resources

- Maven Getting Started Guide
- Building Java Projects with Maven
- Spring Boot Maven Installation
- Maven Transitive Dependencies
- Maven Standard Directory Layout